EE357 Final Report: A NAS system based on Raspberry Pi

Hongtao Lin, Jingyu Cui, Qianyang Peng

Abstract

With the developing of Internet, there is exponentially more and more data available. However, download can sometimes be time-consuming. Besides, some approach like P2P download is potentially harmful to the hard driver and may result in a decreased life-span. To solve these problem, we proposed and implemented a multi-functional network-attached storage system based on Raspberry Pi, a mini-sized but powerful computer.

This system will takeover the downloading tasks, and can transmit the downloaded files via FTP. Additional features includes file sharing and personal homepage, which is designed for university students.

The report is organized as follows: first, our motivation is stated and the related work including open-source softwares we used in this system is introduced. Then we will have a look at our system: how it works and the way it is implemented. Finally, our conclusion is drawn followed by our work distribution, future work and acknowledgement.

1 MOTIVATION

Regularly, individuals have a tendency to match others' behaviors, so it is more possible for people within a local network to ask for similar contents. For example, in dormitories, things always happen that several roommates require same softwares of some specific version, same newly released films, or same computer games. Based on these fact, we hope to build a dormitory oriented NAS system with functions mainly including high-speed and energy-saving file sharing, download and storage.

According to the Wikipedia, **network-attached storage (NAS)** is a file-level computer data storage server connected to a computer network providing data access to a heterogeneous group of clients. NAS is specialized for serving files either by its hardware, software, or configuration. It is often manufactured as a computer appliance a purpose-built specialized computer.

The Raspberry Pi is a series of credit card-sized single-board computers. They are cheap, fast, energy-saving, and suitable to build a lightweight server. So we are to implement and extend out functions based on a Raspberry Pi. Here we choose the newest model, Raspberry Pi 3 as our main hardware.

2 RELATED WORK

Our system is based on a bunch of open-source libraries and softwares. The most basic ones are LNMP, which refers to Linux, Nginx, MySQL and PHP. LNMP is a structure widely used in web applications all over the world and our whole system is based on it. All the softwares and programs of our project is run on this structure.

We chose to use VNC (Virtual Network Computer) for remote control, which is a free open source software for remote control designed for UNIX and Linux.

For user's homepages, we choose **Wordpress** https://cn.wordpress.org/ to implement the functions. Wordpress is an open-source blog platform based on PHP and is light-weight enough to be run on a Raspberry-pi.

Our FTP server is based on **vsftpd** https://security.appspot.com/vsftpd.html, which is the abbreviation of "very secure FTP daemon". As its name, one of its greatest feature is its high level of security, and is a free open-source software. Despite being small for purposes of speed and security, it provide solutions for applications like virtual users.

Another important function of our system is to faciliate users to download resources into their local storage with remote control. By doing this, if a user wants to prepare him for a movie after work, he can simply log in to our system, remotely download from the internet.

Of course, a default browser can support HTTP requests to resources, but this standard way of http request for resources has its limitation. It's much fragile to unexpected interruptions. (just think of downloading a large file using browsers like Chrome.) Another inherit defect is the lack of support for other ways of fetching resources, such as using BT, magnet link and metalink. In concerns that many large files are stored in web with BT seeds or magnet links, as well as the inherit advantages to download these files with pausable methods, we decide our system should support BT download and magnet links.

Aria2 https://aria2.github.io/ is such a well-defined open source software that perfectly fit into UNIX system. It supports HTTP/HTTPS, FTP, SFTP, BitTorrent and Metalink. Also, it support various configurations like maximum download speed, multi-connection etc. It's a huge library with thousands of C-like code. Also, it's lightweighted and with high speed than most of the BT downloaders.

But here's a defect for this software. For a user-friendly system, we might not want a system that can only be connected through UNIX commands. For users who want to get rid of lengthy and complex commands as soon as possible, this system is the least possible choice.

Fortunately, there are several implemented web-uis built for aria2. That's where YAAW https://github.com/binux/yaaw comes into play. It built up a system that bridge aria2 with front-end user. For the program running on server side, it uses JSON-RPC (a json-formatted remote procedure call) to evoke certain functions in aria2. JSON-RPC is a remote procedure control protocol encoded in JSON. It is a very simple protocol (and very similar to XML-RPC), defining only a handful of data types and commands. And aria2 supports JSON-RPC formatted requests. For user, the system becomes extremely simple and easy to use: All they need to do is to copy the link and see how the downloading progress goes on.

But for an application for home/dorm users, security issues such as authentication and authorizarion should be taken into consideration. We should not allow ANY web visitor to successfully operate on the remote side, thus a log in system is required. Also, even if a user is allowed into system, we may not want other members to peek at any files in our own domain. So certain machanism should be applied to provide local security. Driven by these needs, our system provides a log-in system behind the YAAW application as well as a machanism to prevent users to peek into others' file system.

With multiple users supported in our system, one interesting function is to *share* files with other people. Our system provides both HTTP share link and BT seed generation. This is a unique advantage our system have over all systems mentioned above. We use BitTorrent http://www.bittorrent.com/ as our BT server.

3 SYSTEM OVERVIEW

3.1 Hardware Structure

Our hardware system mainly contains the components below:

1. A *Raspberry Pi Model 3 B*
2. A SD card
3. An mobile hard disk drive
4. An usb hub

The hardware are organized as below:

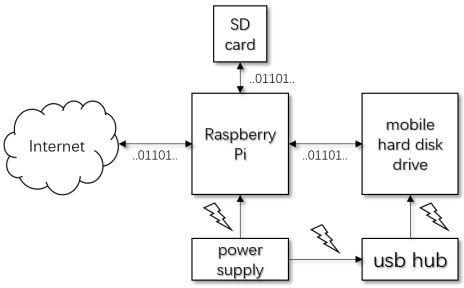

Figure 1: Hardware organization

In this system, the SD card is served as the storage of our Raspberry Pi computer. The operation system, servers and MySQL database is stored on SD card for quick and frequent access. The hard disk, on the other hand, is used to save user files as it is having huge capacity.

The Raspberry Pi computer is directly powered up by a adapter. Although it is having several USB ports, it cannot be used directly as a power supply for mobile disk. So an usb hub is needed.

There are many ways to control the computer. The traditional way is that you can plug a keyboard and mouse onto it, and link it to a monitor by HDMI. However it is somehow not convenient. A more easy method is to directly monitor and control it by some telecontrol software, here we choose *VNC Viewer* to do this.

3.2 User System

Done with the set up of hardware, we would come to another basic function, which is register. Just like most other system with user account supported, our system provides: register, login and a default homepage once logged in successfully. Due to tightly restricted time schedule, update personal information, as well as reset password are not implemented in our system. But it's a predictable and relatively easy task.

3.3 Homepage Customization

After register, you will be directed to the page to initialize your homepage. Here you should set your site name, username, password and email. After setting these stuff you will be ready to use your homepage.

Figure 2: Homepage initialization

Log in your account and press the Wordpress button to enter your homepage. Here you should log in again using the username and password set above, as Wordpress is an independent system and you can give others administrator account to let them edit your webpage without knowing your system account.

Having logged in as an administrator, you can do anything you like to your homepage. You can change your website style, publish or delete photos or articles and so on. Also, you can create other accounts and let your friends to help you manage your website.

The homepage module is powered by *Wordpress*.

3.4 File Download

Another big part for the system is to download resources from web. Our YAAW application

support HTTP download, BitTorrent download as well as Magnet link. When a user click on ”add” button, a detailed configuration page would pop up, one can either use a download link in the website, or a uploaded BT seed for downloading. For configuration, file name and path can be changed as expected.

The rest of the page shows neccesary information about previous and current download events. Also, a virtual file system is visualized for user so that he can grasp a general view of his own storage space. For security issues, other users' storage space are filtered out.

3.5 File Sharing

Considering our system as a connection tool between different families or dorms, our system support file sharing via multiple ways. When clicking on sharable files, a tool tip would pop up which offers two sharing tools. One is HTTP sharing. If clicked, a url will be generated and using this url, one can easily download the specified file using browsers. It's quite similar to share links in BaiduYun. Due to tight schedule of the project, currently only public share link is supported, but note that this share link can be easily generalized to one with password required, which is just a simple modification to database and user interface.

Also, BT sharing is provided in case of large file. If clicked, a generated bt seed will be downloaded to your local PC, using this seed, you can share them as you wish.

3.6 FTP Files Management

FTP is another important component of our NAS system. Each user will have two FTP accounts with different authority. Here we refer a “admin user” to that with full read and write authority to the directory, and a “public user” to that only permitted to download. Both accounts are assigned automatically as the user finishes his register. In this section, a webpage to show the account information is first introduced. Then we will specify the direction and authority of the two accounts respectively.

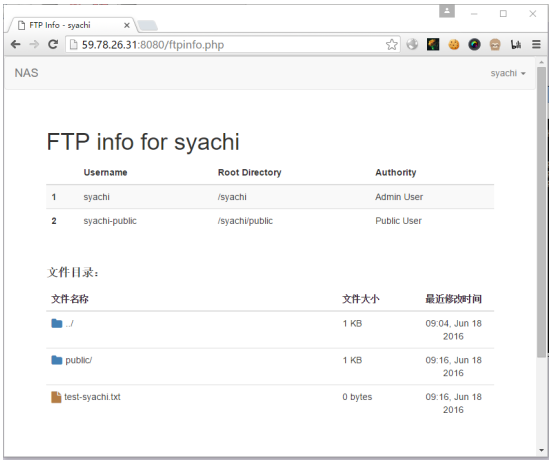

Figure 3: FTP information page

For simplicity, we assume that the username is [name] and the corresponding password used to register our system is [password].

3.6.1 FTP Information Page

After log in, a user will come to the index with three buttons. The third Button named FTP will lead the user to his FTP information page. The information page is consisted with two parts, as is shown in Figure 3.6.1 .

The first part is the account detail for the two FTP users. In our system, admin user has the same username and password as the system account. That is to say, the username and password for admin user is also [name] and [password] respectively. The root directory for admin user is [user]/. For the public user, the username is [name]-public and the password is “public”. Notice that all the public users have the same password. This will not cause security problems because the public user is restricted to the public directory in the admin directory. The public user can only see and download files in [user]/public/, but can not get access to the admin directory, or perform other operations in the public directory.

The second part is the directory detail. A user can view the files and folders in his root directory. Notice that this do not provide methods to operate on files, and is only a way to go through the admin directory.

Figure 4: FTP admin directory structure

3.6.2 FTP Admin User

One can access to his ftp with admin account, and is able to perform various kind of operations within his own directory. Notice that no user can visit the parent directory of his own root, meaning that other users' directory is invisible thus inaccessible. The directory structure is shown in Figure 3.6.2 , which is accessed by FileZilla. The user can also upload or download files. Extra operation including deleting and renaming files are also available. Files that the user want to share can be moved to the public directory, where public users can view and download them.

3.6.3 FTP Public User

If the admin user share the public username with others, they are permitted to visit only the public directory under the admin root. Likewise, the public user is not granted the authority to change root and have any access to the admin root. This can be regarded as a kind of protection of the admin user's privacy and file security. Besides, the public user can only download files from the public root, and other operations are strictly restricted.

4 DETAILED WORK

4.1 System Installation

First, we format the SD card and write the image file with the Raspbian system into the card. Connect a monitor to our raspberry via HDMI, as well as mouse and keyoard. We will first configure the Ethernet connection. Our IP is 59.78.26.31 with mask 255.255.255.0, and gateway 59.78.26.254. The configuration process is simple and will thus not be included in our report. We can try ping command to ensure that the network is available. For remote access, we should install a server on our Raspberry Pi. After installation VNC server, run the initialization command to set the password. Then we can start VNC server with customized options.

```
$ sudo apt-get install tightvncserver
$ tightvncserver
...set password...
$ vncserver -geometry 1280x720 :1
```

We can also write a small file to start VNC after booting. The file we use is downloaded from the Internet. After downloading, change the owner and grant privileges. Then update this file into the startup services.

```
$ wget http://www.penguintutor.com/
    otherfiles/tightvncserver-init.txt
$ mv tightvncserver-init.txt /etc/init.d
    /tightvncserver
$ chown root:root /etc/init.d/
    tightvncserver
$ chmod 755 /etc/init.d/tightvncserver
$ update-rc.d tightvncserver defaults
```

Then we can access to this Raspberry Pi without monitor. The remaining tasks are all done using remote control.

4.2 FTP Server

An FTP server will mainly provide access to a certain directory via FTP protocol. With a admin identity, the user can read and write to the FTP directory. Detailed process includes: uploading, downloading, creating or deleting files or folders, renaming files or folders. In this section, we will mainly focus on installing vsftpd and have a look into the configuring file of vsftpd.

For installation, use the command *apt-get* as listed below to fetch and install. After installation, the FTP should be accessible using the username and password of the raspberry user (pi, raspberry).

```
$ sudo apt-get install vsftpd
```

Notice that after installing, the default root directory for the user is */home/pi/*, which is the home directory for our user pi. We have to set the root to our mobile storage device. The hard disk is mounted at */media/pi/raspberry/* and is with ext4 file system. For example, we

want to set root to *ftp/* directory under the hard disk directory.

The configuration file is saved at */etc/veftpd.conf*, and the syntax is in the form $[key] = [value]$. For most of the keys with 0-1 option, the corresponding value is YES or NO. Line comments begins with a "#".

```
# allow local user to write
write_enable=YES
# disable users to visit directories
    other than root
chroot_local_user=YES
# allow writing with disabled chroot
allow_writeable_chroot=YES
# disable anonymous users to access
anonymous_enable=NO
# configuration of local root
local_root=/media/pi/raspberry/ftp/
```

The meaning of each line is given by Line comments. With this modified configuration file, we can access to the directory on the mobile storage.

4.3 HTTP Server

The HTTP server follows the classic LNMP structure, which includes Linux, Nginx, MySQL and PHP. They are all open-source and light-weight so that we can choose it to build out HTTP server.

You can start this work once you have installed the operating system. Use *apt-get* to install the softwares:

```
$ sudo apt-get install nginx php5-fpm
    php5-cli php5-curl php5-gd php5-
    mcrypt php5-mysql php5-cgi mysql-
    server
```

After installation, we configured the nginx settings in */etc/nginx/sites-sites-enabled/* and */etc/nginx/sites-sites-available/*. Notice that port 80 is blocked by SJTU's firewall, here we set 8080 as the port nginx listen to. Then, we set the "root" folder and some basic reactions to different outside requests.

Now we can write our first *index.php* and put it into the "root" folder set above. By visiting http://59.78.26.31:8080 you can visit the website we've just created!

4.4 User System

The implementation of user system is relatively simple. It requires the cooperative work of both back-end logic and front-end interface. Using php+mysql as the standard tools, we create a table in database stroing informations about user, which are: automatically generated user id, user name, and password. Note that password stored in database must be encrypted so that malevolent attackes cannot sniff at these critical messages easily. A standard way is to use *md5* to salt the password. *md5* is a encrypting algorithm that can be hardly decoded if the oassword is relatively complex. For users who want to log in, the input sequence will undergo the same md5 process to be compared with the message in database. This simple method guarantees a high-level security for users.

Note that the following application is initially an independent project, in order to assign different user with different interface, we have to do several adjustments, which will be mentioned in the following section when needed.

4.5 Personal Homepage

Here we use wordpress to implement our personal homepage. Wordpress is an open-source blog platform based on PHP. It is fast, extendable, and having lots of powerful plugins. Using this platform we can let users freely customize and manage their homepages without much effort.

Apparently we should allocate an identical homepage for each user. So after new user's registration, server should make a new copy of wordpress package, create a new MySQL database for this new copy, and make corresponding initializations to the wordpress config files. We implemented this part of function in python script and call this script file after register operation in *register.php*.

Now we will introduce how to install a wordpress platform.

First download and extract wordpress package, use the derived folder as the original source.

```
$sudo wget http://cn.wordpress.org/
    latest.tar.gz
$sudo tar xzvf latest.tar.gz
```

When a new user is created, the python script receives the username and password, and then use them to create a new database in MySQL.

```
cur.execute('CREATE␣DATABASE␣if␣not␣
    exists␣%s'%(username+'_wp'))
cur.execute('GRANT␣ALL␣PRIVILEGES␣ON␣%s
    .*␣TO␣"%s"@"localhost"IDENTIFIED␣BY␣
    "%s"'%((username+'_wp'), (username+'
    _wp'), password))
cur.execute('FLUSH␣PRIVILEGES')
```

Then the python script make a copy of the original source to the user's personal folder.

```
os.mkdir(userdir)
shutil.copytree("/srv/www/webservice/
    public_html/src/wordpress",userdir+"
    /homepage")
```

And lastly, the script edit the config file of wordpress to tell it the database name, username and password.

```
lines=open(oriwpconfig,'r').readlines()
flen=len(lines)-1
for i in range(flen):
    if 'database_name_here' in lines[i]:
        lines[i]=lines[i].replace('
            database_name_here',(
            username+'_wp'))
    if 'username_here' in lines[i]:
        lines[i]=lines[i].replace('
            username_here',(username+'
            _wp'))
    if 'password_here' in lines[i]:
        lines[i]=lines[i].replace('
            password_here',password)
open(wpconfig,'w').writelines(lines)
```

So that we successfully created a new wordpress platform. After registration we will direct users to the wordpress setting page and let themselves to set their homepage, as is described in *3.3*.

4.6 FTP Virtual User

The aim of FTP virtual user is to allow different users with varied authority (public, admin) to visit their corresponding directory. And we have to pass this work to PHP in order to realize registering FTP user via web application. In this section, we will first see how to set up the environment and create virtual users with command line. Then an approach using PHP will be introduced.

4.6.1 Configuring Virtual User

There are two approaches to create virtual users, regarding where the users' information is saved. One method save information in a local file, while ours will use MySQL to maintain accounts. With MySQL and vsftpd already installed on our system, we only need to install an authority module named PAM.

```
$ sudo apt-get install pam-mysql
```

Then we will create a local FTP user in our raspbian system, and grant the authority of its home directory to this newly created user. This system user is used to read records from MySQL.

```
$ sudo mkdir /home/vsftpd vsftpd
$ sudo useradd   d  /home/vsftpd vsftpd
$ sudo chown -R vsftpd /home/vsftpd
```

If we want to start virtual user service on vsftpd, we should again modify the configuration file mentioned in section 4.2. We add the following lines into */etc/veftpd.conf*.

```
# allow virtual user
guest_enable=YES
# set the corresponding system user
guest_username=vsftpd
# specify the pam authorization file
pam_service_name=/etc/pam.d/vsftpd
```

With this config file, vsftpd will seek for the pam authorization file and check if the requesting user is a legitimate one. Then we need a table in MySQL for users' accounts. Since we already have a database named "app", we will create the table in this database.

```
$ mysql -u root -p
mysql> use app;
mysql> create table ftpuser(id int NOT
    NULL,
mysql> username varchar(255) NOT NULL,
    password varchar(255) NOT NULL
mysql> public int NOT NULL, PRIMARY KEY
    (username) );
mysql> grant all privileges on app.
    ftpuser to vsftpd@localhost
    identified by 'raspberry';
mysql> public int NOT NULL, PRIMARY KEY
    (username) );
```

The key "public" is a 0-1 indicator meaning if the user is a public user (0) or an admin user (1). Key "id" correspond to the user id when registering our system, which is mentioned in section 4.4. And we can try to insert some users for testing like (pi, raspberry) with a admin identity, and (pi-public, public) with a public identity. Then we come to modify the pam authorization file, which can be

found in */etc/pam.d/vsftpd*. We comment out all the contents and add the following lines:

```
auth required pam_mysql.so user=vsftpd
    passwd=raspberry host=localhost db=
    app table=ftpuser usercolumn=
    username passwdcolumn=password crypt
    =0
account required pam_mysql.so user=
    vsftpd passwd=111111 host=localhost
    db=app table=ftpuser usercolumn=
    username passwdcolumn=password crypt
    =0
```

In our testing process, there are something wrong with the encrypted authentication using pam, so we set crypt=0, which means passwords are plain texts. With this config file, pam will check the database if the user exists with system user vsftpd. Then the users are able to log in using FTP client. But we are only half way done. Then we will specify different authority for users. Recall the vsftpd configuring file, we are to modify it again. Add one line to set the directory saving configuration files for individual users.

```
user_config_dir=/etc/vsftpd_user_conf
```

And we will edit config files for users. These files share the same syntax with *veftpd.conf*, and can be regarded as additional content to the main conf file. The file should share the same name with the corresponding user.

```
$ sudo mkdir /etc/vsftpd/
    vsftpd_user_conf
$ sudo touch /etc/vsftpd/
    vsftpd_user_conf/pi
$ sudo nano /etc/vsftpd/vsftpd_user_conf
    /pi
$ sudo mkdir /media/pi/raspberry/ftp/pi
$ sudo mkdir /media/pi/raspberry/ftp/pi/
    public
$ sudo chmod 777 -R /media/pi/raspberry/
    ftp/pi
add:
local_root=/media/pi/raspberry/ftp/pi
# share the same authority as the local
    user
virtual_use_local_privs=YES
$sudo nano /etc/veftpd/vsftpd_user_conf/
    pi-public
add:
local_root=/media/pi/raspberry/ftp/pi/
    public
```

Now the file structure is clear: the public user can only access to a folder within the root directory of the admin user, with a limited authority.

4.6.2 Using PHP to Takeover

Finally we need to let PHP takeover our work and register FTP users automatically rather than manually. To complete this task, we need to grant a system root privilege to the HTTP server. Type "visudo" in the terminal to modify the file */etc/sudoers*. Find a line beginning with "www-data", which is the user for Nginx. Replace with:

```
www-data ALL=(ALL) NOPASSWD:ALL
```

The three "ALL"s stands for hosts, target users, and the permitted operations. "NOPASSWD" means that the user can execute commands without entering password. Then PHP is ready to run commands with local root authority. Part of the PHP script for creating a admin user is shown below, this correspond to the procedure mentioned previously.

```
<?php
mysql_query("INSERT␣INTO␣ftpuser(
    username,password,id,public)␣VALUES
    ('$user','$pass',$id,0)");
$user_config = "/etc/vsftpd_user_conf/".
    $user;
system("touch␣$user_config",$status1);
system("chmod␣777␣$user_config",$status2
    );
$user_root = "/media/pi/raspberry/ftp/".
    $user;
system("sudo␣mkdir␣$user_root",$status3)
    ;
system("sudo␣chmod␣777␣$user_root",
    $status4);
$status5 = file_put_contents("
    $user_config","local_root=$user_root
    ".PHP_EOL."virtual_use_local_privs=
    YES");
system("sudo␣chown␣root:root␣
    $user_config",$status0);
?>
```

The creation of a public user is similar, and will thus not be detailed in our report. With the PHP script, we can create FTP account for our user as he/she finished register.

4.7 BitTorrent Server

A bittorrent server is needed for users to share files via seed torrents. First we make the installation of BitTorrent, and we need to start a tracker. Here the tracker is set to port 6969.

```
$ /usr/local/bin/ bttrack.py --port 6969
    --dfile dstate >/dev/null 2>&1 &
```

To keep a stable running environment and prevent the frequent file change in user folders, we keep a independent folder to keep the torrents and source files. So an user can only share a file not having the same name with any current file, and the system will make a copy of the file to the torrent folder, and create a torrent file.

```
# btmakemetafile.py /where/is/your/file/
    filename http://localhost:6969/
    announce
```

Now all the source files to share and the corresponding torrents is in the torrent folder. So we can start a downloader to periodically scan the torrent folder and initialize the download source.

```
# btlaunchmany /where/is/your/torrent/
    folder
```

Share the .torrent file and you can make other users to download what you share!

4.8 Multi-protocol Downloader

Using YAAW mentioned in related work, we could easily download HTTP and bt seed from web. Our work is to do several modifications to make YAAW well-fitted into ours.

The first thing is to make user specific configuration possible. Since different users are virtually seperated from each other, two modifications should be done in order to persist this relation.

1. **Download Seperation**

   When a user raises up a download request, we examine his intended destination directory. If the directory is valid (i.e., within his own file system), this request is qualified and file will be downloaded. Otherwise, we would raise up a warning, indicating invalid request.

2. **Visulization and Filtering**

   For user's conveinence, we visulize the file system within YAAW. This is done by requesting neccessary informations about all files and directories within his default directory, using an interactive table to visualize these information. When a user click at a directory, the information will be updated.

   But the convenience brings up some insecurities. In order to prevent user from peeking at other's directory, each time we evaluate the requested directory, and filter out those directories which he's not authenticated.

Since YAAW has already implemented the interface to aria2, downloading files is no longer a problem. Additional work is to make these files sharable. In the web interface, for each file, we create an associated tool tip. When a user click on a file, the tool tip will pop up with two choices for HTTP and BT share, respectively. The link is associated with the certain file id, and can be passed to backend to complete certain logic.

For HTTP share link, we intend to generate a short link like BaiduYun did. Our short link is generated by first convert the file path into *md5* encrypted strings, then use *base64* encoding to further process the string. Since the short link cannot be recovered, we hav e to store this mapping into database. When the short link is requested, it will be remapped to a file path by querying through database. Downloading file from php is to simple read binary texts from it and flush them to output buffer.

For BT seed generation, it would require some hard work to do. As mentioned in previous subsection, we can simply generate a bit torrent seed using a line of command. The rest of work seems to be reading from the file and dump it to buffer. But the real process is obscure and comfusing actually. The problem is the inconsistency between file being read and file being acutally sent to user's PC. The file size is a mismatch, which is ridiculous at first sight. After searching for related bugs and several trial and errors, we start to speculate the problem highly related with how a BT seed is encoded.

If you ever open a bt seed using text editor, you might be comfused with incomprehesable letters and possibily garbled message. Understanding the true meanings behind these code

d8:announce30:http://localhost:6969/announce13:creation
datei1466226895e4:infod6:lengthi11853e4:name4:test12:piece
lengthi262144e6:pieces20:'äàå*¯ìYÅ¡_ÄSYN§^ `Foee

Figure 5: An example of BT seed content

is useful for generating bt seed by ourselves as well as downloading the seed to your PC.

Simply put, a BT seed is a special encoding for a json-like dictionary. Here's an example converted from 4.8:

```
{
 "announce": <url>,
 "creation date": 1466080045,
 "info":
 {
  "length": 412435,
  "name": aria2,
  "piece length": 262144,
  "pieces": <Hashed code>
 }
}
```

Note the garbled message in field "pieces" is a SHA1 hash code for a block of data in files. Every block with given length will be hased into a 20 bytes SHA1 code. It's useful for sanity checking when transfered. But usually, the hashed code cannot be decoded successfully into readable characters. So the backend (here is php) or frontend (here is javascript) system will process the text in a wrong way. And these garbled message may be translated into other some special characters indicating the error. That's why the file received is not the same with the original one.

With the source of bugs figured out, we realized that we have to disable the processing of text when it's transfered to front-end system. By doing so, the original binary code is successfully dumped to local PC, and thus the BT seed.

5 CONCLUSION

Generally, from hardware to software, we designed and implemented a powerful, energy-saving and user-friendly system. The system can be used for both personal purpose and in small local area networks. With a cost of less than 700 and power consumption of less than 10W, you can get a 24-hour homepage server, a remote downloader, a FTP server and a full-featured file sharing system. You can use it only for your self, also you can use it together with your family or roommates.

Apart from using it as an online disk to storage your files and share them with others, there are many other cool applications. Suppose these scenarios. If you want to download something but you are outside, or you are playing games and download will make your computer lag, or the file to download is extremely large and will take you several days to download, use our NAS system is your best choice! You can start or manage the download anywhere you have access to the Internet, and do your download quietly, separately, and using less than one tenth power of your computer.

Another usage is if you and your roommates want to have personal homepages for applications of study abroad, this is also a perfect choice. It is free, stable, multiuser supportive, and much more energy-saving than a real server! Also, you are completely free to customize your page and can do whatever you want here.

What's more, if you are abroad in a country where BT download is illegal, doing BT download may cause you big trouble. However, if you have our device in China, you can use remote downloader to complete the BT download, and use FTP to get your file downloaded to your computer, thus avoiding you from all the illegal operations.

That is our system. It's exciting, isn't it?

6 FUTURE WORKS

This project is having a very strong extendability. In other words, almost anything supports Linux and ARM architecture can be implemented in our system. However, on our current work, there are still something needs

further research.

The most important one is that now little attention is paid on resource allocation. That is one user may take most of the bandwidth, leaving little bandwidth to other users. And one user may use most of the storage room, leaving no spare space for others to use. Although our system is designed for environment people know and trust each other, implementing some constraints for such possible greedy users never a bad choice.

Also, all the functions now is mainly based on a single set of system. If there are more than one system working together, there may have more cool applications such as distributed storage and high-speed file transmission. As we have only one system and time is limited, such part of research is left as our future work.

And lastly, what if there is some error with our disk and our important files are damaged? For example we can apply a RAID (Redundant Arrays of Independent Disks) on our system and do regular file backup, thus making our system more stable.

There is no limit to the study, and that is the most charming part for us.

7 WORK DISTRIBUTION

Qianyang Peng:

1. Designed the hardware structure;
2. Configured the LNMP framework and HTTP server;
3. Implemented the features about personal homepage;
4. Configured the BitTorrent server and implemented the P2P file sharing system.

Hongtao Lin:

1. the main structure of user system, including register, log in and main entrance logic.
2. the installation and modification of YAAW system, including user-specific download path configuration, visulization of file system, sharing files via HTTP link and BT seed.

Jingyu Cui:

1. Installed and configured vsftpd system.
2. Enabled virtual users.
3. Enabled controlling FTP accounts using PHP.
4. Installed the system and enabled remote controlling.
5. Maintained the hardware, including backup and recovery.

8 ACKNOWLEDGEMENT

Our project received great support from Professor Weijia Jia. Prof. Jia expressed his great interests at times when we first proposed the idea to him. We discussed the project with him several times, when he gave us many pragmatic advices and farforseeing ideas. Also, he gave us a strong financial support in buying neccessary devices: including a raspberry pi 3 model B and a hard disk for storage. Also, we thank our TA for his well-considered schedule of project.